

第2回：LLM APIの基礎

Pythonから gpt-6-luna を呼び出す

今日のゴールと流れ

APIへの入力と、返ってきた出力を説明できる。

時間	内容
10分	前回の復習、モデルとAPI
20分	HTTP、JSON、APIキー
20分	Pythonコードの読み方
30分	呼び出しと比較の演習
10分	利用量、エラー、提出

サービスの利用とAPIの利用

チャット画面では、人が入力し、画面に答えが表示されます。

APIでは、Pythonが入力を送り、Pythonが結果を受け取ります。

受け取った結果を画面に表示したり、他の処理へ渡したりできます。

チャット製品の契約とAPIの利用枠は、同じとは限りません。

講師の案内に従ってAPI利用環境を準備します。

HTTPとJSONの復習

要素	LLM APIでの意味
POST	入力を送り、回答の生成を依頼
認証ヘッダー	利用するアカウントのキーを渡す
JSONのリクエスト	モデル、指示、質問など
JSONのレスポンス	生成結果、状態、利用量など

Python SDKは、この通信を呼びやすくするライブラリです。

2種類のAPI

第3回以降は、APIが2か所に登場します。

API	呼ぶ側	受け取る側
自作の <code>/api/ask</code>	ブラウザ	自分のFastAPI
Responses API	自分のPython	外部のLLMサービス

今日は下の通信だけを扱います。Web画面はまだ作りません。

秘密情報の保管

APIキーを知る人は、利用権限を使える場合があります。

- Codespaces Secretsの名前は `OPENAI_API_KEY`
- このリポジトリへのアクセスを許可する
- 設定後はCodespacesを停止・再開する
- キーの値を画面、ログ、提出物に貼らない

漏れた場合はキーを無効化・再発行します。Gitから消すだけでは不十分です。

.envによる設定

Codespaces Secretsを使わない場合の方法です。

```
cd session02/exercise  
cp .env.example .env
```

`.env` をエディタで開き、キーを設定します。

APIを使うときは `DEMO_MODE=false` に変更します。

`.env` はGitの対象外。環境変数が既にあれば、その値を優先します。

今回のファイル

ファイル	役割
<code>one_shot.py</code>	質問を入力し、結果を表示
<code>llm.py</code>	APIを呼び出す
<code>settings.py</code>	モードとモデル名を読む
<code>.env.example</code>	秘密を含まない設定例

読み順は `one_shot.py`、`ask_plain()`、`make_client()` です。

入口のコード

```
question = input('質問: ').strip()
result = ask_plain(question)
print(result['answer'])
print('入力トークン:', result['input_tokens'])
print('出力トークン:', result['output_tokens'])
```

関数に質問を渡し、戻り値の辞書から必要な項目を取り出します。

`strip()` は文字列の前後の空白を除きます。

クライアントの作成

```
from openai import OpenAI  
  
client = OpenAI(timeout=30.0, max_retries=0)
```

- `OPENAI_API_KEY` はSDKが環境変数から取得
- `timeout` は応答待ちの時間の設定
- `max_retries=0` は自動再試行をしない設定

教材では、意図しない再送による追加呼び出しを避けます。

Responses APIの呼び出し

```
response = client.responses.create(  
    model=MODEL, # settings.pyではgpt-6-luna  
    reasoning={'effort': 'none'},  
    instructions='日本語で簡潔に教えてください。',  
    input=question,  
    max_output_tokens=600,  
    store=False,  
)
```

教材は短い応答の観察から始めるため、推論設定を `none` にします。

指定した項目の意味

項目	意味
model	呼び出すモデルのID
instructions	アプリが与える指示
input	今回の質問や参考情報
max_output_tokens	出力の上限。推論トークンも含む
store	Responses APIでの応答保存の指定

`store=False` だけで、あらゆるサービス側ログが無くなるわけではありません。
秘密情報は送信しません。

応答の取り出し

```
answer = response.output_text  
input_tokens = response.usage.input_tokens  
output_tokens = response.usage.output_tokens
```

SDKの `output_text` で回答のテキストを取り出します。
教材は `status` が完了か、本文が空でないかも確認します。
途中で切れた回答を、成功として表示しないためです。

実習1：API呼び出しの穴埋め

1. `llm.py` の `TODO` を探す
2. 呼び出し部分のコメントを外す
3. `raise NotImplementedError(...)` を削除する
4. `python one_shot.py` を実行する
5. 「FastAPIを2文で説明して」と入力する

デモではAPI部分を通りません。穴埋めの確認は講師指定のAPI環境で行います。

実習2：指示と質問の比較

同じ質問のまま、指示だけを変更します。

試行	指示の例
A	初学者向けに2文で説明する
B	箇条書きで3項目にまとめる
C	専門用語には短い説明を添える

内容・形式・入力と出力のトークン数を記録します。
同じ入力でも同じ結果を保証するとは限りません。

利用料金の考え方

概算は、入力と出力のトークン数に、それぞれの単価を掛けます。

例：単価が100万トークン当たりの表示なら、

$$\begin{aligned} & \text{入力数} \div 1,000,000 \times \text{入力単価} \\ & + \text{出力数} \div 1,000,000 \times \text{出力単価} \end{aligned}$$

実際にはキャッシュ等の条件も関係します。

料金表と利用状況画面を講師と確認し、数値は開講時に更新します。

よくあるエラー

状況	確認すること
キーが見つからない	Secretsの名前・対象リポジトリ・再起動
認証エラー	キーの有効性。値そのものは共有しない
モデルを使えない	<code>gpt-6-luna</code> の利用権限・モデル名
利用制限	レート制限・利用枠・請求設定
待ち時間が長い	ネットワーク・入力の長さ

モデルを勝手に変更せず、講師に状況を伝えます。

Pythonの例外

```
try:  
    result = ask_plain(question)  
except ValueError as error:  
    print(error)
```

`try` で処理し、失敗したときは `except` へ進みます。
次回は失敗をHTTPステータスと画面のメッセージに変換します。

確認問題

1. ブラウザのJavaScriptにキーを書いてよい？
2. 出力を短くしても、入力トークンは料金に関係する？
3. `instructions` と `input` は何を分けている？
4. デモモードのトークン数0は、無料のLLMを使った意味？

提出物と次回

提出：変更した `llm.py` と `worksheet.md` のGitHub URL。
APIキー、`.env`、APIの生の例外全文は提出しません。

```
git status
git add session02/exercise/llm.py session02/exercise/worksheet.md
git commit -m "第2回: API呼び出しと比較"
git push
```

次回は、同じ処理をWeb画面から呼び出します。